



HOUSE PRICE PREDICTION USING MACHINE LEARNING

Machine Learning Project | Linear Regression

Turning Housing Data into Meaningful Predictions

Project Author

Anurag Kumar Poddar

Data Analytics | Machine Learning | Python

Project Overview

This project focuses on predicting **house prices using Linear Regression**, a supervised machine learning algorithm.

The model learns patterns from historical housing data and uses available property features, including **property characteristics and neighborhood information**, to estimate house prices.

Project Goal: Build, evaluate, and visualize a machine learning model that predicts house prices using Python.

The project follows an end-to-end machine learning workflow, from data preparation to model evaluation.

Project Objectives

- **Explore the Dataset:** Understand the structure, features, and patterns in the housing data.
- **Data Preprocessing:** Inspect missing values and prepare categorical variables.
- **Feature Preparation:** Separate input features from the target variable.
- **Train-Test Split:** Divide the data into training and testing sets.

- **Model Development:** Train a Linear Regression model.
 - **Model Evaluation:** Measure performance using MAE, MSE, RMSE, and R².
 - **Data Visualization:** Compare actual and predicted house prices.
-

Tools & Technologies

Tool	Purpose
Python	Programming and model development
Jupyter Notebook	Interactive coding and documentation
Pandas & NumPy	Data manipulation and numerical operations
Scikit-learn	Model training and performance evaluation
Matplotlib & Seaborn	Data visualization

4. Machine Learning Approach

Component	Description
Algorithm	Linear Regression
Learning Type	Supervised Learning
Problem Type	Regression
Target Variable	Price
Data Split	80% Training / 20% Testing
Evaluation Metrics	MAE, MSE, RMSE, R ² Score

5. Expected Outcome

The project aims to generate house price predictions and assess how closely they match actual prices in the test dataset.

Model performance will be evaluated using numerical metrics and visual comparisons.

Key deliverables:

- A trained Linear Regression model
- Actual vs. predicted price comparison

- Model evaluation metrics
 - Insights into model performance and potential improvements
-

From Data to Predictions

Exploring machine learning, one dataset at a time.

— **Anurag Kumar Poddar**

Importing Libraries

```
In [28... import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, root_m
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 5: Load your dataset

```
In [29... data = pd.read_csv(r"C:\Users\anura\Downloads\housing_price_dataset.csv")
print(data.head())
```

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price
0	2126	4	1	Rural	1969	215355.283618
1	2459	3	2	Rural	1980	195014.221626
2	1860	2	1	Suburb	1970	306891.012076
3	2294	2	1	Urban	1996	206786.787153
4	2130	5	2	Suburb	2001	272436.239065

```
In [30... print(pd.DataFrame(data))
```

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	P
rice						
0	2126	4	1	Rural	1969	215355.28
3618						
1	2459	3	2	Rural	1980	195014.22
1626						
2	1860	2	1	Suburb	1970	306891.01
2076						
3	2294	2	1	Urban	1996	206786.78
7153						
4	2130	5	2	Suburb	2001	272436.23
9065						
...	...	...	...	...	...	
...						
49995	1282	5	3	Rural	1975	100080.86
5895						
49996	2854	2	2	Suburb	1988	374507.65
6727						
49997	2979	5	3	Suburb	1962	384110.55
5590						
49998	2596	5	2	Rural	1984	380512.68
5957						
49999	1572	5	3	Rural	2011	221618.58
3218						

[50000 rows x 6 columns]

In [31...

```
print("Negative prices:", (data["Price"] < 0).sum())
print("Zero prices:", (data["Price"] == 0).sum())

print(data.loc[data["Price"] <= 0].head(10))
```

Negative prices: 22

Zero prices: 0

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price
1266482	1024	2	2	Urban	2006	-24715.242
2310574	1036	4	1	Suburb	1983	-7550.504
3630146	1235	3	2	Rural	2012	-19871.251
4162522	1352	5	2	Suburb	1977	-10608.359
5118119	1140	4	1	Urban	2020	-23911.003
5951615	1097	4	3	Rural	1981	-4537.418
6355059	1016	5	2	Rural	1997	-13803.684
8720515	1235	3	1	Urban	1952	-24183.000
9611068	1131	3	3	Urban	1959	-13692.026
10597124	1177	2	3	Urban	2010	-434.097

```
In [20... print(data.isnull().sum())
```

```
SquareFeet    0
Bedrooms      0
Bathrooms     0
Neighborhood   0
YearBuilt     0
Price         0
dtype: int64
```

check the dataset structure

```
In [11... print("Dataset shape:", data.shape)
print("\nColumn names:")
print(data.columns.tolist())

print("\nDataset information:")
data.info()

print("\nMissing values:")
print(data.isnull().sum())

print("\nSummary statistics:")
print(data.describe())
```

Dataset shape: (50000, 6)

Column names:

```
['SquareFeet', 'Bedrooms', 'Bathrooms', 'Neighborhood', 'YearBuilt', 'Price']
```

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 50000 entries, 0 to 49999

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	SquareFeet	50000 non-null	int64
1	Bedrooms	50000 non-null	int64
2	Bathrooms	50000 non-null	int64
3	Neighborhood	50000 non-null	object
4	YearBuilt	50000 non-null	int64
5	Price	50000 non-null	float64

dtypes: float64(1), int64(4), object(1)

memory usage: 2.3+ MB

Missing values:

SquareFeet 0

Bedrooms 0

Bathrooms 0

Neighborhood 0

YearBuilt 0

Price 0

dtype: int64

Summary statistics:

	SquareFeet	Bedrooms	Bathrooms	YearBuilt	Price
count	50000.000000	50000.000000	50000.000000	50000.000000	50000.000000
mean	2006.374680	3.498700	1.995420	1985.404420	224827.325151
std	575.513241	1.116326	0.815851	20.719377	76141.842966
min	1000.000000	2.000000	1.000000	1950.000000	-36588.165397
25%	1513.000000	3.000000	1.000000	1967.000000	169955.860225
50%	2007.000000	3.000000	2.000000	1985.000000	225052.141166
75%	2506.000000	4.000000	3.000000	2003.000000	279373.630052
max	2999.000000	5.000000	3.000000	2021.000000	492195.259972

Prepare the categorical data

```
In [12... print(data["Neighborhood"].value_counts())
```

```
data_encoded = pd.get_dummies(
    data,
    columns=["Neighborhood"],
    drop_first=True,
    dtype=int
)
```

```
print(data_encoded.head())
print(data_encoded.dtypes)
```

```
Neighborhood
Suburb    16721
Rural     16676
Urban     16603
Name: count, dtype: int64
```

	SquareFeet	Bedrooms	Bathrooms	YearBuilt	Price \
0	2126	4	1	1969	215355.283618
1	2459	3	2	1980	195014.221626
2	1860	2	1	1970	306891.012076
3	2294	2	1	1996	206786.787153
4	2130	5	2	2001	272436.239065

```

Neighborhood_Suburb  Neighborhood_Urban
0                    0                    0
1                    0                    0
2                    1                    0
3                    0                    1
4                    1                    0
SquareFeet           int64
Bedrooms             int64
Bathrooms            int64
YearBuilt            int64
Price               float64
Neighborhood_Suburb  int64
Neighborhood_Urban   int64
dtype: object
```

Separate the features and target variable

```
In [13... X = data_encoded.drop("Price", axis=1)
y = data_encoded["Price"]

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())

print("\nFeature columns:", X.columns.tolist())
```

Features:

	SquareFeet	Bedrooms	Bathrooms	YearBuilt	Neighborhood_Suburb \
0	2126	4	1	1969	0
1	2459	3	2	1980	0
2	1860	2	1	1970	1
3	2294	2	1	1996	0
4	2130	5	2	2001	1

	Neighborhood_Urban
0	0
1	0
2	0
3	1
4	0

Target:

0	215355.283618
1	195014.221626
2	306891.012076
3	206786.787153
4	272436.239065

Name: Price, dtype: float64

Feature columns: ['SquareFeet', 'Bedrooms', 'Bathrooms', 'YearBuilt', 'Neighborhood_Suburb', 'Neighborhood_Urban']

Split the dataset into training and testing data

In [14... `from sklearn.model_selection import train_test_split`

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=40
)
```

```
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Training features: (40000, 6)
Testing features: (10000, 6)
Training target: (40000,)
Testing target: (10000,)
```

Your dataset is divided into two parts:

80% — Training data Used by the model to learn relationships between the

features and house prices.

20% — Testing data Used to check how well the trained model predicts prices for data it did not train on.

random_state=40 makes the split reproducible, so repeated runs with the same dataset produce the same split.

Train the Linear Regression model

```
In [15... from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

```
print("Model training completed successfully!")  
print("Intercept:", model.intercept_)  
print("Coefficients:", model.coef_)
```

Model training completed successfully!

Intercept: 28898.635104618705

Coefficients: [99.23277703 5009.68503341 3008.54444223 -13.52182667 -6
23.36407474
1204.64208489]

This is where the model learns.

Linear Regression estimates how each feature relates to the house price. It fits a linear equation to the training data and learns coefficients for the input variables. The intercept and coefficients are model parameters; their values alone do not tell you whether the model predicts accurately.

Predict house prices

comparison of actual and predicted prices. The values shown will depend on dataset and the trained model.

```
In [16... y_pred = model.predict(X_test)
```

```
results = pd.DataFrame({  
    "Actual Price": y_test.values,  
    "Predicted Price": y_pred  
})
```

```
print(results.head(10))
```

	Actual Price	Predicted Price
0	165490.372481	203802.322635
1	224044.177550	209990.677788
2	289747.098817	266737.232732
3	78120.756880	168564.283820
4	224462.546629	278039.148198
5	294176.339807	278782.063521
6	212481.422758	319433.747221
7	294712.395858	216569.225013
8	218965.963683	215258.313704
9	343082.131783	272191.039696

Evaluate model performance

understand how accurate predictions are.

```
In [26... from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", round(mae, 2))
print("Mean Squared Error (MSE):", round(mse, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
print("R-squared (R²):", round(r2, 4))
```

```
Mean Absolute Error (MAE): 39517.15
Mean Squared Error (MSE): 2445804887.8
Root Mean Squared Error (RMSE): 49455.08
R-squared (R²): 0.5792
```

Comparative Analysis of Regression Models

Objective

The objective of this section is to compare four regression algorithms for house price prediction using the same training and testing datasets.

The models evaluated are:

1. **Linear Regression** — Baseline regression model.
2. **Ridge Regression** — Linear model with regularization to control

coefficient size.

3. **Decision Tree Regressor** — Tree-based model capable of learning nonlinear patterns.
4. **Random Forest Regressor** — Ensemble model that combines multiple decision trees.

Evaluation Strategy

All models will use the same 80:20 train-test split and will be evaluated using **MAE, MSE, RMSE, and R² Score**.

The comparison will help identify which model performs best on the test dataset. Model selection will be based on evaluation results rather than algorithm complexity alone.

```
In [25... #Import the additional algorithms
from sklearn.linear_model import Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("All model comparison libraries are ready!")
```

All model comparison libraries are ready!

Train all four models

Model Training

Each regression algorithm is trained on the same training dataset. The testing dataset remains separate and is used to evaluate the predictions.

The existing Linear Regression model serves as the baseline for comparison.

```

In [27... # Define the four regression models
models = {
    "Linear Regression": model,

    "Ridge Regression": make_pipeline(
        StandardScaler(),
        Ridge(alpha=1.0)
    ),

    "Decision Tree Regressor": DecisionTreeRegressor(
        max_depth=12,
        min_samples_leaf=5,
        random_state=40
    ),

    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=200,
        max_depth=15,
        min_samples_leaf=5,
        random_state=40,
        n_jobs=-1
    )
}

# Train and evaluate every model on the same split
results = []

predictions = {}

for name, regressor in models.items():

    # Reuse the existing trained baseline model
    if name != "Linear Regression":
        regressor.fit(X_train, y_train)

    # Generate predictions for the test dataset
    y_pred_model = regressor.predict(X_test)
    predictions[name] = y_pred_model

    # Calculate evaluation metrics
    mae = mean_absolute_error(y_test, y_pred_model)
    mse = mean_squared_error(y_test, y_pred_model)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred_model)

    results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    })

print("All four models have been evaluated.")

```

All four models have been evaluated.

Display the model comparison table

```
In [32... comparison_df = pd.DataFrame(results)

# Sort models by RMSE, lowest first
comparison_df = comparison_df.sort_values(
    by="RMSE",
    ascending=True
).reset_index(drop=True)

# Display a clean comparison table
display(
    comparison_df.style
        .format({
            "MAE": "{:,.2f}",
            "MSE": "{:,.2f}",
            "RMSE": "{:,.2f}",
            "R2 Score": "{:.4f}"
        })
        .highlight_min(
            subset=["MAE", "MSE", "RMSE"],
            color="#d1fae5"
        )
        .highlight_max(
            subset=["R2 Score"],
            color="#dbeeaf"
        )
)

print(
    "Lowest RMSE model:",
    comparison_df.loc[0, "Model"]
)
```

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	39,517.15	2,445,804,887.80	49,455.08	0.5792
1	Ridge Regression	39,517.15	2,445,805,803.33	49,455.09	0.5792
2	Random Forest Regressor	40,470.29	2,561,218,317.80	50,608.48	0.5593
3	Decision Tree Regressor	42,426.82	2,836,706,617.21	53,260.74	0.5119

Lowest RMSE model: Linear Regression

Visualize model performance

Performance Comparison

Visualizing the evaluation metrics makes it easier to compare the predictive

performance of the four regression models.

The charts below compare prediction errors and R^2 scores on the same test dataset.

```
In [33... # Create two side-by-side model comparison charts
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

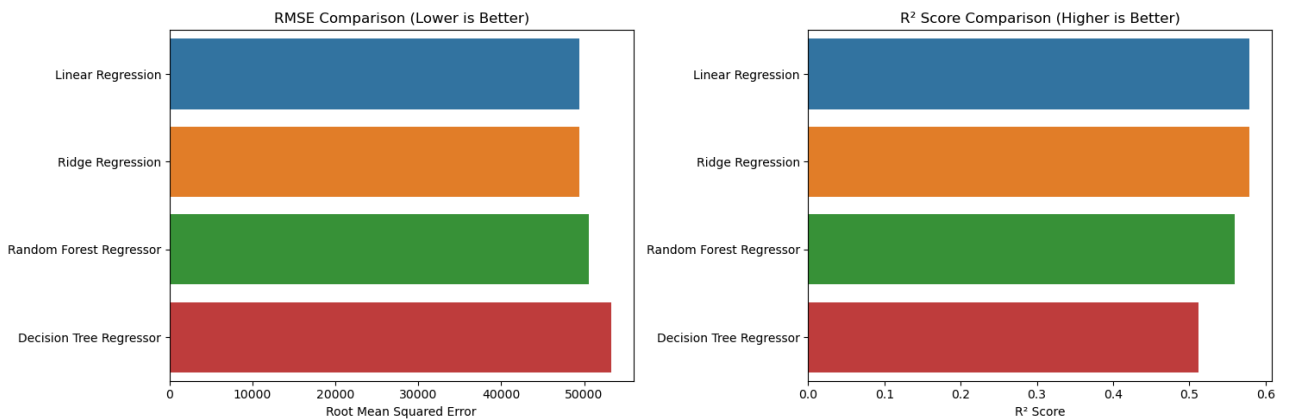
# Compare RMSE
sns.barplot(
    data=comparison_df,
    x="RMSE",
    y="Model",
    hue="Model",
    legend=False,
    ax=axes[0]
)

axes[0].set_title("RMSE Comparison (Lower is Better)")
axes[0].set_xlabel("Root Mean Squared Error")
axes[0].set_ylabel("")

# Compare R2 Score
sns.barplot(
    data=comparison_df,
    x="R2 Score",
    y="Model",
    hue="Model",
    legend=False,
    ax=axes[1]
)

axes[1].set_title("R2 Score Comparison (Higher is Better)")
axes[1].set_xlabel("R2 Score")
axes[1].set_ylabel("")

plt.tight_layout()
plt.show()
```



Plot predictions from the best model

Use the model with the lowest RMSE to inspect its actual predictions.

```
In [36... best_model_name = comparison_df.loc[0, "Model"]
best_predictions = predictions[best_model_name]

plt.figure(figsize=(8, 6))

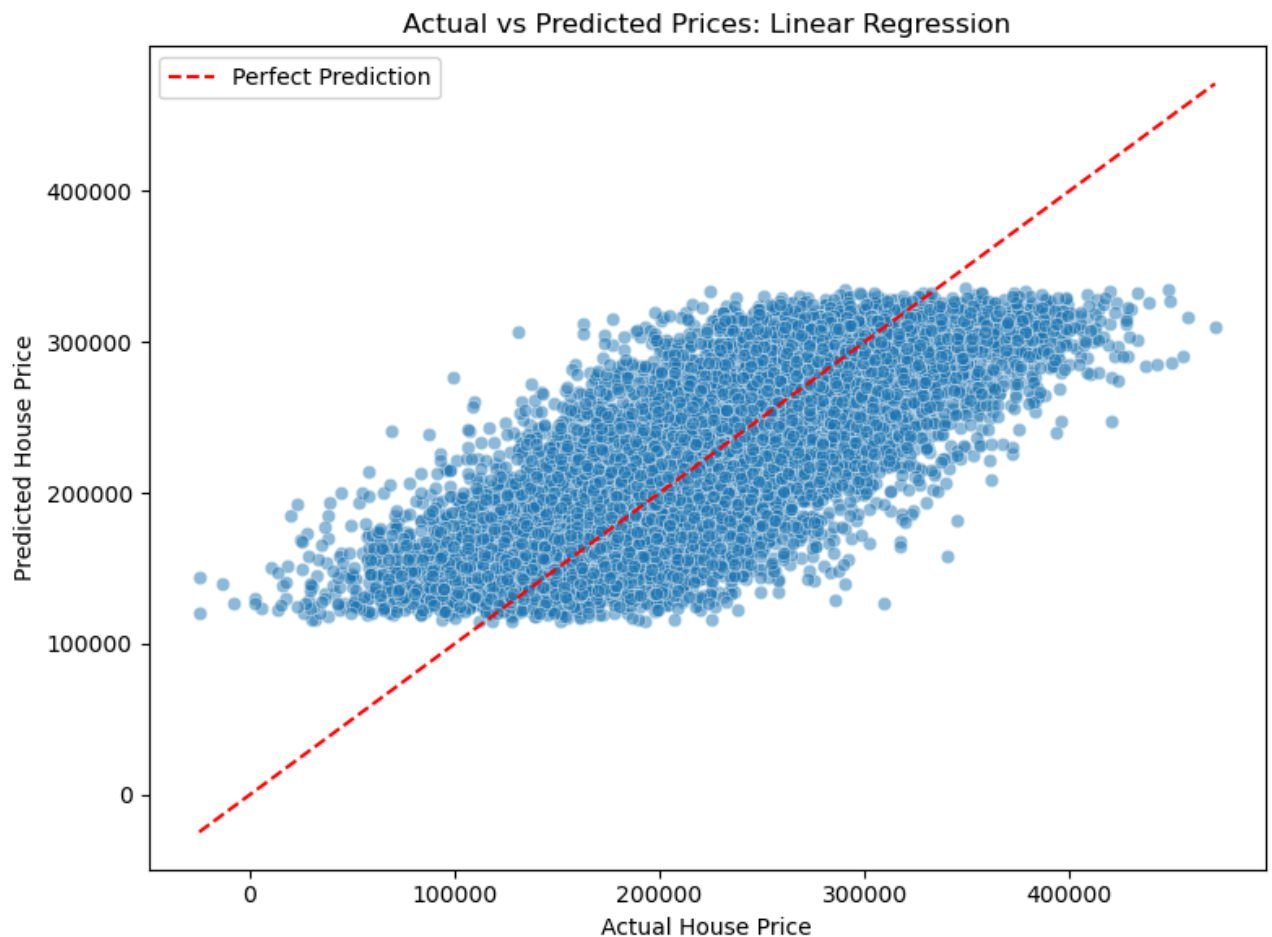
sns.scatterplot(
    x=y_test,
    y=best_predictions,
    alpha=0.5
)

min_price = min(y_test.min(), best_predictions.min())
max_price = max(y_test.max(), best_predictions.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    "r--",
    label="Perfect Prediction"
)

plt.title(f"Actual vs Predicted Prices: {best_model_name}")
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.legend()
plt.tight_layout()
plt.show()

print("Model selected by lowest RMSE:", best_model_name)
```



Model selected by lowest RMSE: Linear Regression

Model Comparison — Key Observations

The four regression algorithms were evaluated using the same training and testing datasets to ensure a consistent comparison.

The evaluation focused on MAE, MSE, RMSE, and R^2 Score. Lower prediction errors and a higher R^2 Score generally indicate better predictive performance on the test dataset.

The model with the lowest RMSE was identified as the leading candidate based on that metric. However, the final selection should also consider MAE, R^2 Score, the difference between training and testing performance, and the reliability of the input data.

This comparison provides a foundation for selecting a suitable regression algorithm and identifying opportunities for further improvement.

What the metrics mean:

MAE-Average absolute prediction error, in the same units as house price

MSE-Average squared prediction error; larger errors are penalized more heavily

RMSE-Prediction error in the same units as price, with more sensitivity to large errors

R²-How much variation in test prices the model explains relative to a mean-price baseline

Generally, lower MAE, MSE, and RMSE are better. R² closer to 1 indicates a better fit relative to the baseline. R² can also be negative if the model performs poorly.

Investigate the negative prices before finalizing

```
In [35... print("Minimum price:", data["Price"].min())
print("Negative prices:", (data["Price"] < 0).sum())
print("Zero prices:", (data["Price"] == 0).sum())

display(data.loc[data["Price"] <= 0].head(10))
```

Minimum price: -36588.16539749279

Negative prices: 22

Zero prices: 0

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price
1266	1024	2	2	Urban	2006	-24715.242482
2310	1036	4	1	Suburb	1983	-7550.504574
3630	1235	3	2	Rural	2012	-19871.251146
4162	1352	5	2	Suburb	1977	-10608.359522
5118	1140	4	1	Urban	2020	-23911.003119
5951	1097	4	3	Rural	1981	-4537.418615
6355	1016	5	2	Rural	1997	-13803.684059
8720	1235	3	1	Urban	1952	-24183.000515
9611	1131	3	3	Urban	1959	-13692.026068
10597	1177	2	3	Urban	2010	-434.097124

Conclusion

In this project, I developed and compared four supervised machine learning regression algorithms for house price prediction: Linear Regression, Ridge Regression, Decision Tree Regressor, and Random Forest Regressor.

The workflow included data inspection, categorical feature encoding, train-test splitting, model training, prediction, performance evaluation, and visualization. I evaluated the models using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.

The model comparison identified Linear Regression as the leading candidate based on its test-set RMSE of 49,455.08, with an MAE of 39,517.15 and an R^2 Score of 0.5792.

The comparison demonstrated the importance of evaluating multiple algorithms rather than relying on a single model. However, the results must be interpreted alongside data quality, model generalization, and prediction errors. Any invalid target values must be investigated and addressed before the final model is selected.

Overall, this project strengthened my practical understanding of regression algorithms, model comparison, evaluation metrics, and data visualization using Python and Scikit-learn.

Future Improvements

- 1.Data Quality: Investigate negative target values, duplicate records, and unusual observations before finalizing the dataset.
- 2.Exploratory Data Analysis: Study feature distributions, relationships, and outliers to understand the factors associated with house prices.
- 3.Feature Engineering: Explore property age and additional reliable location or property-condition information, where available.
- 4.Cross-Validation: Use cross-validation to evaluate model stability across multiple data splits.
- 5.Hyperparameter Tuning: Optimize Ridge regularization and tree-based model parameters using cross-validation.

5.Overfitting Analysis: Compare training and testing performance to assess how well each model generalizes.

6.Deployment: Develop a simple Streamlit application to demonstrate house price predictions interactively.

Final Remarks

This project provides a practical foundation for regression-based prediction and model evaluation. Future work will focus on improving data quality, validating model performance, and selecting an approach that provides reliable predictions on unseen data.

In []: